

TFP KILL TRACKER

Full Technical Documentation

Version 1.2 · 30 March 2026

1. Overview

TFP Kill Tracker is a fully client-side, single-page web application that fetches live kill statistics from a public Google Sheets CSV export and renders them as a sortable, filterable leaderboard. There is no back-end server; all processing — CSV parsing, aggregation, filtering, and DOM rendering — occurs in the browser.

1.1 Technology Stack

Layer	Technology	Notes
Front-end	Vanilla HTML / CSS / JavaScript (ES6+)	No frameworks or build tools
Fonts	Google Fonts – Oswald & Open Sans	Loaded via CDN
Data Source	Google Sheets public CSV export	Fetches at page load via fetch()
Deployment	GitHub Actions → GitHub Pages	Automatic on push to main / claude/*
Documentation	python-docx	This document

1.2 Key Design Principles

- Zero dependencies — no npm, no bundler, no framework.
- Two-file front-end — index.html (structure + CSS) and tracker.js (all JavaScript); split for CSP compliance on GitHub Pages.
- Stateless server — all state is in-memory JavaScript variables; nothing is persisted server-side.
- Re-aggregate on every filter change — guarantees all derived metrics stay consistent.
- Collapsible sections — each of the 11 content sections can be independently collapsed; state persists in localStorage.

2. Architecture

2.1 File Structure

File	Purpose
index.html	HTML structure and inline CSS; references tracker.js
tracker.js	All dashboard JavaScript — parsing, aggregation, filtering, rendering
favicon.svg	Browser tab icon (SVG crosshair graphic)
unit_overrides.json	Manual unit classification corrections applied on top of auto-classification
ImportScript	Google Apps Script that imports OCAP .json.gz mission logs into Google Sheets
generate_docs.py	python-docx script that regenerates this documentation file
README.md	User-facing feature overview and deployment guide
.github/workflows/	GitHub Actions CI/CD pipeline for GitHub Pages deployment

2.2 Data Flow

The application follows a strict linear pipeline on every filter change:

1. **FETCH** — GET request to Google Sheets CSV export URL (CSV_URL constant in tracker.js). Also fetches unit_overrides.json in parallel.
2. **PARSE** — Raw text split into lines; each line processed by parseCSVLine() to handle quoted fields with embedded commas.
3. **AGGREGATE (full)** — buildAggregates() loops all rows once and builds aggPlayers, a keyed object of career totals per player.
4. **UNIT CLASSIFY** — classifyPlayerUnits() analyses squad co-occurrence to assign units (2nd USC, CNTO, PXG, TFP); manual overrides from unit_overrides.json applied on top.
5. **FILTER** — applyFilters() re-loops rawRows applying four independent filters (event type, unit, player selection, mission selection, Zeus status), building filteredPlayers and filteredRows.
6. **RENDER** — Thirteen render functions consume filteredPlayers / filteredRows and write HTML to specific DOM targets.

Steps 5–6 re-execute on every user interaction (filter toggle, pill click, column sort, section expand). Steps 1–4 run once at page load.

2.3 Global State Variables

Variable	Type	Default	Purpose
rawRows	Array<Object>	[]	Every CSV row parsed into {header: value} objects
aggPlayers	Object	{}	Full career aggregates keyed by player name
playerUnits	Object	{}	Unit assignment keyed by player name (from auto-classification + overrides)
filteredPlayers	Array<Object>	[]	Aggregated stats for the current filter set
filteredRows	Array<Object>	[]	Raw rows matching current filter (used by section renders)

Variable	Type	Default	Purpose
selectedPlayers	Set null	null	null = all players; Set = explicit selection
selectedMissions	Set null	null	null = all missions; Set = explicit selection
selectedUnit	string null	null	Active unit filter pill ("2nd USC", "CNT0", "PXG", "TFP", or null)
showJointOps	boolean	true	Include Joint Op events
showRegularEvents	boolean	true	Include regular op events
zeusFilter	string	"all"	"all" "no-zeus" "zeus-only"
infSortCol	number	2	Active sort column index for infantry table
infSortAsc	boolean	false	Sort direction for infantry table
vehSortCol	number	2	Active sort column index for vehicle table
vehSortAsc	boolean	false	Sort direction for vehicle table
unitSortCol	number	2	Active sort column index for unit leaderboard
unitSortAsc	boolean	false	Sort direction for unit leaderboard
vkSortCol	number	0	Active sort column index for vehicle kills section
vkSortAsc	boolean	false	Sort direction for vehicle kills section
wpSortCol	number	0	Active sort column index for weapon leaderboard
wpSortAsc	boolean	false	Sort direction for weapon leaderboard
currentModalPlayer	string	""	Player name currently shown in detail modal
RELEASE_DATE	string	YYYY-MM-DD	Hardcoded release date shown in the page footer

3. Data Source

3.1 CSV Configuration

The application fetches from the URL defined by the constant `CSV_URL` near the top of `tracker.js`. To point the tracker at a different sheet, update this constant. The sheet must be published publicly ("File → Share → Publish to web → CSV").

3.2 Expected Column Schema

One row per player per mission. Columns produced by the ImportScript:

Index	Column Name	Description
0	Source File	Filename encoding date (YYYY_MM_DD_...) and mission name
1	Mission	Human-readable mission name
2	World	Map/terrain name
3	Username	Player name (used as aggregate key)
4	Side	Faction (e.g. BLUFOR, OPFOR)
5	Group	Squad/group name; "zeus" group triggers Zeus classification
6	Role	In-game role string; used for role normalisation and top-role detection
7	Kills (On Foot)	Infantry kills
8	Deaths (On Foot)	Infantry deaths
9	K/D (On Foot)	Ignored — recomputed from kills/deaths
10	Teamkills (On Foot)	Friendly fire kills (infantry)
11	Shots (On Foot)	Rounds fired on foot
12	Hits Taken (On Foot)	Times hit while on foot
13	Shots/Kill (On Foot)	Ignored — recomputed
14	Avg Kill Dist On Foot (m)	Average kill distance (on foot)
15	Longest Kill On Foot (m)	Longest single kill distance (on foot)
16	Kills (In Vehicle)	Vehicle kills
17	Deaths (In Vehicle)	Deaths while in vehicle
18	K/D (In Vehicle)	Ignored — recomputed
19	Teamkills (In Vehicle)	Friendly fire from vehicle
20	Vehicle Kills (On Foot)	Vehicles destroyed while on foot
21	Vehicle Kills (In Vehicle)	Vehicles destroyed while mounted
22	Shots (In Vehicle)	Rounds fired from vehicle
23	Hits Taken (In Vehicle)	Times vehicle was hit
24	Shots/Kill (In Vehicle)	Ignored — recomputed
25	Avg Kill Dist In Vehicle (m)	Average kill distance (vehicle)
26	Longest Kill In Vehicle (m)	Longest single kill distance (vehicle)
27	Top Weapon	Weapon used for the longest kill; shown on award cards

Index	Column Name	Description
28	Weapon Kills (JSON)	JSON object: {"WeaponName [MagType]": killCount, ...}; quotes preserved during parse
29	Suicides	Self-inflicted deaths
30	Time Played (s)	Seconds present in the mission
31	Distance Run (km)	Total distance run from OCAP movement data; teleport steps (>100 m) excluded

3.3 CSV Parsing

parseCSVLine(line) handles RFC-4180 compliant CSV including quoted fields that contain commas or embedded double-quotes (escaped as ""). The parser is character-by-character and maintains a boolean inQuote state. Quotes within quoted fields are unescaped before storage.

3.4 Mission Date Extraction

Mission names are expected to contain an ISO date in parentheses, e.g. "Operation Nightfall (2024-03-15)". The missionDate(missionName) function applies the regex `/((\d{4}-\d{2}-\d{2}))\)/` and reformats the result to DD/MM/YYYY for display. If no date is found the original string is returned unchanged.

```
missionDate("Op Nightfall (2024-03-15)") → "15/03/2024"
```

3.5 Joint Op Detection

isJointOp(filename) determines whether a mission should be classified as a Joint Op. The logic parses the YYYY_MM_DD prefix from the filename, determines the day-of-week, then checks whether that Saturday is the last Saturday of its calendar month, or whether the date is a Sunday immediately following the last Saturday.

```
// filename format: "2024_03_30_MissionName.sqm"
// Returns true if date is last Saturday or last-Saturday+1 (Sunday) of the month
```

4. Aggregation Engine

4.1 buildAggregates()

Called once after the CSV is fetched. Loops rawRows and builds aggPlayers — a dictionary keyed by player name. Each entry is initialised with zero values on first encounter, then accumulated across every row.

Computed fields per player:

- kdFoot / kdVeh — kills ÷ deaths (Infinity if deaths = 0, 0 if kills = 0)
- spkFoot / spkVeh — shots ÷ kills (null if kills = 0)
- avgDistFoot / avgDistVeh — weighted mean: $(\sum \text{avgDist} \times \text{kills}) \div \sum \text{kills}$
- missionCount — cardinality of the missions Set
- topRole — normalizeRole() applied to Group; role with highest count wins
- longestKillWeapon — weapon column from the row that contained the max distance

4.2 Player Object Schema

```
{
  name: string,
  missions: Set<string>,          // unique mission names
  worlds: Set<string>,           // unique map names

  // ■■ Infantry ■■
  killsOnFoot: number,
  deathsOnFoot: number,
  tkOnFoot: number,
  shotsOnFoot: number,
  hitsOnFoot: number,
  suicides: number,
  distanceRun: number,           // km
  timePlayed: number,           // seconds
  maxLongestFoot: number,        // metres
  avgDistFoot: number,          // weighted mean (metres)

  // ■■ Vehicle ■■
  killsInVeh: number,
  deathsInVeh: number,
  tkInVeh: number,
  vehKillsFoot: number,          // vehicles destroyed while on foot
  vehKillsVeh: number,          // vehicles destroyed while in vehicle
  shotsInVeh: number,
  hitsInVeh: number,
  maxLongestVeh: number,
  avgDistVeh: number,

  // ■■ Derived ■■
  kdFoot: number,
  kdVeh: number,
  spkFoot: number | null,
  spkVeh: number | null,
  missionCount: number,

  // ■■ Meta ■■
  weaponKills: { [weapon: string]: number },
  longestKillWeapon: string,
  roleCounts: { [role: string]: number },
  topRole: string | null,
  topRoleCount: number,
  unit: string | null, // "2nd USC" | "CNT0" | "PXG" | "TFP" | null
```

```
missionRows:      Array<CSVRowObject>,  
}
```

4.3 Role Normalisation

normalizeRole(role) strips unit prefixes and cosmetic suffixes before role counting, so that "SL Alpha @player" and "SL Alpha" are counted as the same role.

- Removes @username suffixes
- Strips Alpha/Bravo/Charlie/Delta/Echo/Foxtrot squad letters
- Removes numbered squad prefixes (S1, S2 ...)
- Removes colour names (Red, Blue, Green, Yellow ...)
- Removes NATO phonetic identifiers (Alpha, Bravo ... Foxtrot)

5. Filter System

5.1 Filter Pipeline

`applyFilters()` is the central function that rebuilds `filteredPlayers` from `rawRows`. Filters are applied in sequence; a row must pass all active filters to contribute to the output aggregation.

Step	Filter	State Variable	Condition
1	Event Type	<code>showJointOps / showRegularEvents</code>	<code>isJointOp(srcFile)</code> must match the selected event type
2	Player	<code>selectedPlayers</code>	<code>null = all</code> ; else <code>row.Username</code> must be in the Set
3	Mission	<code>selectedMissions</code>	<code>null = all</code> ; else <code>row.Mission</code> must be in the Set
4	Zeus	<code>zeusFilter</code>	Applied post-aggregation: checks if player has any <code>Group="zeus"</code> row

5.2 Pill Availability Cascade

Available pills are recomputed by `refreshPills()` after every filter change to prevent showing players or missions that have no data under the current filter set.

1. `getEventFilteredRows()` returns rows matching the event-type filter.
2. Available missions = unique Mission values in those rows.
3. Available players = unique Username values in rows that also match the selected missions.
4. Both pill lists are re-rendered against the search input substring filter.

5.3 Player Pills vs Mission Pills

Aspect	Player Pills	Mission Pills
Renderer	<code>renderCareerPills()</code>	<code>renderSelectablePills()</code>
Left-click	Toggle player selection (<code>filterToPlayer</code>)	Toggle mission selection
Right-click	Open full career page (<code>openCareerPage</code>)	No action
Multi-select	Yes — multiple players can be active simultaneously	Yes — multiple missions can be active simultaneously
Active state	<code>pill.active</code> class (dark background)	<code>pill.active</code> class (dark background)
null state	<code>null = all players active</code>	<code>null = all missions active</code>
Search	<code>playerSearch</code> input (case-insensitive substring)	<code>missionSearch</code> input (case-insensitive substring)

5.4 Zeus Filter

The Zeus filter is applied after re-aggregation. For each player in the filtered aggregation, the system checks whether any of their `rawRows` has `Group (lowercased) === "zeus"`.

Value	Behaviour
"all"	Include all players regardless of Zeus status
"no-zeus"	Exclude players who have at least one Zeus row in the filtered set
"zeus-only"	Include only players who have at least one Zeus row

6. Rendering Functions

6.1 renderStats()

Renders the top stats bar with aggregate totals across all filteredPlayers.

Card Label	Value Source
Infantry Kills	Σ killsOnFoot
Kills from Vehicles	Σ killsInVeh
Total Deaths	Σ (deathsOnFoot + deathsInVeh)
Teamkills	Σ (tkOnFoot + tkInVeh)
Vehicles Destroyed	Σ (vehKillsFoot + vehKillsVeh)
Players	filteredPlayers.length
Missions	unique mission count across filtered rows
Average K/D	$(\Sigma \text{ killsOnFoot}) / (\Sigma \text{ deathsOnFoot})$, formatted to 2 d.p.

6.2 renderChart()

Renders a horizontal bar chart of the top 10 infantry killers. Bar widths are proportional to the highest kill count in the visible set. Each bar shows player name and kill count as a label.

6.3 renderLeader() — Award Cards

Award	Icon	Metric	Eligibility	Display
Executioner	■	Most killsOnFoot	killsOnFoot > 0	Name · kills · K/D
Pro Sniper	■	Highest maxLongestFoot	Min 2 kills on foot	Name · distance (m) · avg dist · weapon
Perfect Aim	■	Lowest spkFoot (shots/kill)	Min 3 kills, spkFoot ≥ 1	Name · SPK · kill count
K/D Player	■ ■	Highest kdFoot	Min 3 kills on foot	Name · K/D · kills/deaths

6.4 renderInfantryTable()

Renders the Infantry Combat Stats table. Columns:

#	Column	Key	Default Sort
0	Rank (#)	_rank	—
1	Player	name	alpha
2	Kills	killsOnFoot	★ desc
3	Veh Kills	vehKillsFoot	desc
4	Deaths	deathsOnFoot	desc
5	K/D	kdFoot	desc (green ≥2, red <0.8)
6	TK	tkOnFoot	desc (red if >0)
7	Suicides	suicides	desc
8	Shots	shotsOnFoot	desc
9	Hits Taken	hitsOnFoot	desc
10	Shots/Kill	spkFoot	asc
11	Avg Dist (m)	avgDistFoot	desc

#	Column	Key	Default Sort
12	Longest (m)	maxLongestFoot	desc
13	Missions	missionCount	desc

Rows with `tkOnFoot > 0` receive the `.tk-row` CSS class (light pink background). Rank medals (■■■■) are assigned by `killsOnFoot` position. Clicking a row opens the player detail modal. The ■ icon in the Player column opens the full career page.

6.5 renderVehicleTable()

Renders the Vehicle Combat Stats table. Only players with at least one vehicle kill or death are included. Column structure mirrors the infantry table but for vehicle metrics.

6.6 renderTableHead()

Renders clickable `<th>` elements with sort state indicators. The active sort column shows ▲ (ascending) or ▼ (descending); inactive columns show ■. Clicks call `_sortInf()` or `_sortVeh()` which toggle direction if the same column is clicked twice.

6.7 renderUnitLeaderboard()

Aggregates `filteredPlayers` by unit (2nd USC, CNTO, PXG, TFP, Unknown). Computes per-unit totals for players, kills, deaths, K/D, TK, vehicle kills, kills/player, avg K/D, missions, distance run, and time played. All columns are sortable. Right-clicking raw-count headers toggles per-mission averages (~m mode).

6.8 renderMissionHistory()

Builds a chronological table of every unique mission in `filteredRows`. Per-mission totals: date, world, player count, kills, deaths, K/D, TK, vehicle kills, and top killer (player with highest kills in that mission). Sorted by date descending.

6.9 renderVehicleKillsSection()

Renders a flat list of every vehicle destruction across all `filteredRows`. Parses the Vehicle Kills (JSON) column (an array of {v: vehicle, w: weapon} objects) for each player row and expands them into individual records. Columns: date, mission, vehicle, killer, weapon. All columns are sortable via `window._sortVK()`. If no vehicle kill data exists (missions imported before the JSON column was added), a prompt is shown to reimport with the updated script.

6.10 renderWeaponLeaderboard()

Merges Weapon Kills (JSON) objects across all `filteredRows`. Displays top-50 weapons with total kills, user count, top user, and kill share %. Right-clicking the Total Kills header toggles kills/user mode (amber ~u prefix). A live search input above the table filters rows by weapon name substring.

6.11 renderMapStats()

Groups `filteredRows` by World column. Per-map totals: kills, deaths, K/D, TK, mission count, player count. Sorted by kills descending.

6.12 renderRoleLeaderboard()

Groups `filteredPlayers` by their `topRole` (normalised). Per-role aggregates: player count, total kills, deaths, K/D, avg K/D. Sorted by kills descending.

6.13 renderAttendance()

Builds a per-player attendance table. Columns: player, missions attended, attendance % (relative to total missions in filtered set), first seen date, last seen date, avg time per mission. Sorted by mission count descending.

6.14 renderKdTrend()

Renders an SVG line chart (viewBox 900x220) of K/D ratio over time. Data points are per-mission K/D values grouped by source file date. A dropdown allows switching between Overall, per unit, or individual player views. A dashed yellow reference line marks $K/D = 1.0$. SVG dots show exact values on hover. A "nice number" algorithm targets ~6 Y-axis grid lines regardless of data range.

6.15 renderComparison()

Side-by-side stat comparison between two players selected from dropdowns. Renders a table of all infantry and vehicle metrics; the higher value in each row is highlighted in green. Draws from filteredPlayers for current filter context.

6.16 Section Nav Bar & toggleSection()

A sticky dark tab strip (#sectionNav) sits at the top of the page with one button per section. Clicking a button calls navTo(key), which expands the section if collapsed, re-renders its chart (K/D Trend, Comparison) if it was first rendered while hidden, then smooth-scrolls to the section header. toggleSection(key) toggles the section content div between display:block and display:none and persists the state to localStorage under key "sec_" + key.

7. Player Detail Views

7.1 Player Detail Modal

Triggered by clicking any row in either table. Opens a floating overlay (.modal-overlay.open) showing a full career breakdown for the selected player.

Element	Content
Header	Player name, mission count, active date range, top role
Maximize button (■)	Closes modal, opens full career page for same player
Close button (X)	Removes .open class; also closed by pressing ESC or clicking overlay
Body	buildCareerStatsHTML() output — 4 sections (see §7.3)

7.2 Full Career Page

Triggered by: right-clicking a player pill, clicking the ■ icon on a table row, or clicking the maximize button in the modal.

When the career page opens:

- selectedPlayers is set to a one-element Set for the player.
- applyFilters() is called — all tables and stats re-filter to this player.
- The stats bar, awards row, chart, and filter panel are hidden.
- The career header and career stats panel are shown.
- The page scrolls to the top.

Career page header shows:

- Player name (large, Oswald font)
- Combat Missions count · Active date range (DD/MM/YYYY – DD/MM/YYYY) · Top Role (count)

Clicking "← Back to Leaderboard" calls closeCareerPage(), which resets selectedPlayers to null, re-shows all leaderboard elements, and calls applyFilters().

7.3 buildCareerStatsHTML() — Content Sections

Section	Content
1 — Overall Infantry Stats	Grid of stat cards: Kills, Deaths, K/D, TK, Shots, Hits Taken, Shots/Kill, Avg Dist, Longest Kill, Missions, Suicides
2 — Weapon Kill Breakdown	Top 15 weapons sorted by kill count; proportional bar chart with kill count and percentage of total kills
3 — Best Single Mission	Mission with the highest kill count; shows kills, K/D, TK, and longest kill for that mission
4 — Kill Breakdown by Mission	Table of all missions sorted by kills desc; columns: Mission, Date, Kills, Deaths, K/D, TK, Longest Kill (m)

8. Complete Function Reference

Function	Category	Parameters	Returns	Description
NUM(v)	Utility	v: any	number	Safe parseFloat with 0 fallback.
fmtTime(secs)	Utility	secs: number	string	Formats seconds as "Xh Ym" or "Ym" for display.
normalizeRole(role)	Utility	role: string	string	Strips @username, squad letters, colour/phonetic suffixes from role names.
missionDate(name)	Utility	name: string	string	Extracts (YYYY-MM-DD) from mission name and reformats to DD/MM/YYYY.
isJointOp(filename)	Utility	filename: string	boolean	Returns true if filename date is the last Saturday or its following Sunday of the month.
parseCSVLine(line)	Parsing	line: string	string[]	RFC-4180 CSV parser that handles quoted fields with embedded commas.
buildAggregates()	Aggregation	—	void	Loops rawRows and builds aggPlayers. Called once at load.
classifyPlayerUnits(rows)	Aggregation	rows: Object[]	Object	Analyses squad co-occurrence to assign each player to a unit; requires >=99% dominant weight.
buildUI()	Lifecycle	—	void	Shows content div, hides loader, shows section nav, calls buildFilters() and applyFilters().
buildFilters()	Filters	—	void	Wires all filter UI event listeners (event type, unit pills, reset, search inputs, Zeus dropdown).
getEventFilteredRows()	Filters	—	Object[]	Returns rawRows filtered by showJointOps / showRegularEvents.
refreshPills()	Filters	—	void	Recomputes available missions and players, then re-renders both pill containers.
renderCareerPills(cld, items, sld)	Filters	containerId, items[], searchId	void	Renders player pills. Left-click = filter; right-click = career page.
renderSelectablePills(cld, items, sel, sld, cb)	Filters	containerId, items[], selectedSet, searchId, onSelect()	void	Generic multi-select pill renderer. null selectedSet means "all active".
filterToPlayer(name)	Filters	name: string	void	Toggles player in/out of selectedPlayers; null if set becomes empty.

Function	Category	Parameters	Returns	Description
filterChanged()	Filters	—	void	Calls applyFilters(). Entry point for all filter state changes.
applyFilters()	Filters	—	void	Core pipeline: filter rawRows → re-aggregate → unit/Zeus filter → render all; updates active filter badge.
renderStats()	Rendering	—	void	Renders aggregate stat cards to #statsBar.
renderChart()	Rendering	—	void	Renders top-10 infantry killers horizontal bar chart to #chartBars.
renderLeader()	Rendering	—	void	Renders all Hall of Fame and Hall of Shame award cards.
renderInfantryTable()	Rendering	—	void	Sorts filteredPlayers and writes infantry table rows/header to DOM.
renderVehicleTable()	Rendering	—	void	Sorts vehicle-active players and writes vehicle table rows/header to DOM.
renderTableHead(hId, cols, sCol, sAsc, tId)	Rendering	headId, cols[], sortCol, sortAsc, tableId	void	Renders <th> elements with ▲▼■ sort indicators and right-click tooltips.
sortPlayers(arr, col, asc, cols)	Rendering	arr, colIdx, ascending, cols[]	Object[]	Generic sort; uses cols[col].sortKey; nulls sorted to end.
renderUnitLeaderboard()	Rendering	—	void	Aggregates filteredPlayers by unit and renders the unit leaderboard table.
renderMissionHistory()	Rendering	—	void	Renders chronological mission history table from filteredRows; includes vehicle kill count per mission.
renderVehicleKillsSection())	Rendering	—	void	Expands Vehicle Kills (JSON) rows into a flat sortable table of vehicle destructions (vehicle, killer, weapon).
renderWeaponLeaderboard()	Rendering	—	void	Merges weapon kill JSON across filteredRows and renders weapon leaderboard.
renderMapStats()	Rendering	—	void	Groups filteredRows by World and renders map stats table.
renderRoleLeaderboard()	Rendering	—	void	Groups filteredPlayers by topRole and renders role leaderboard.

Function	Category	Parameters	Returns	Description
renderAttendance()	Rendering	—	void	Computes per-player attendance metrics from filteredRows and renders table.
renderKdTrend()	Rendering	—	void	Renders SVG K/D trend line chart; reads subject from #kdTrendSubject dropdown.
renderComparison()	Rendering	—	void	Renders side-by-side player comparison table; reads names from comparison dropdowns.
toggleSection(key)	Navigation	key: string	void	Toggles section content div; saves state to localStorage.
navTo(key)	Navigation	key: string	void	Expands section, re-renders chart if needed, smooth-scrolls to header.
openPlayerModal(name)	Modal	name: string	void	Opens the detail modal overlay for the given player.
closeModal(e)	Modal	event	void	Closes modal on overlay click or ESC keypress.
openCareerPage(name)	Career	name: string	void	Switches to full-screen career view; sets selectedPlayers and re-filters.
closeCareerPage()	Career	—	void	Restores leaderboard view; clears selectedPlayers and re-filters.
buildCareerStatsHTML(p)	Career	p: PlayerObject	string	Generates HTML for 4-section career stats (infantry overview, weapons, best mission, mission table).
mStat(val, lbl, sub)	Helpers	val, label, subtitle?	string	Returns HTML for a single stat card element.
kdClass(v)	Helpers	v: number	string	"kd-good" if >=2, "kd-bad" if <0.8, else "".
tkClass(v)	Helpers	v: number	string	"tk-cell" if >0, else "".
fmtTime(s)	Helpers	s: number	string	Formats seconds as Xh Ym string.
window._sortInf(col)	Sort	col: number	void	Infantry table header click handler; toggles direction if same column.
window._sortVeh(col)	Sort	col: number	void	Vehicle table header click handler; toggles direction if same column.
window._sortUnit(col)	Sort	col: number	void	Unit leaderboard header click handler.
window._sortWp(col)	Sort	col: number	void	Weapon leaderboard header click handler.

Function	Category	Parameters	Returns	Description
window._sortVK(col)	Sort	col: number	void	Vehicle kills section header click handler; toggles direction if same column.

9. CSS Design System

9.1 CSS Custom Properties (Variables)

Variable	Value	Usage
--charcoal	#424242	Primary dark colour — headers, section titles, table headers
--charcoal-light	#5a5a5a	Hover states, secondary elements
--bg	#f4f4f4	Page background
--white	ffffff	Card and panel backgrounds
--green	#388e3c	Good K/D (≥ 2), positive indicators
--red	#c62828	Teamkills, bad K/D (< 0.8), error states
--gold	#f9a825	Rank #1 medal, top stat highlights
--silver	#9e9e9e	Rank #2 medal
--bronze	#795548	Rank #3 medal
--border	#e0e0e0	Card/panel borders, table cell borders
--row-hover	#f0f0f0	Table row hover background
--tk-bg	#fff3f3	Teamkill row background (light pink)

9.2 Typography

Font Family	Weights	Usage
Oswald	400, 600, 700	Section headings, award card titles, large stat values, player name on career page
Open Sans	400, 600	Body text, table cells, filter labels, pill text
Courier New	400	Code blocks (documentation only)

9.3 Key CSS Classes

Class	Element	Purpose
.stat-card	<div>	Top stats bar card; flex-grow 1, min-width 110px
.filter-panel	<div>	White rounded panel containing all filter controls
.pill		Filter pill; border-radius 20px; cursor pointer
.pill.active		Dark background (#424242) / white text — active selection
.award-card	<div>	Award card base; rounded; flex column
.ac-kills / .ac-longest / .ac-spk / .ac-kda	<div>	Award card colour variants (background differs per award)
.kd-good	<td>	Green text for K/D ≥ 2
.kd-bad	<td>	Red text for K/D < 0.8
.tk-cell	<td>	Red text for teamkill count > 0
.tk-row	<tr>	Light pink (#fff3f3) row background for players with TKs

Class	Element	Purpose
.rank-gold / .rank-silver / .rank-bronze	<td>	Medal emoji colouring for top-3 ranks by kills
.modal-overlay	<div>	Full-screen semi-transparent overlay; .open class shows it
.career-header	<div>	Career page top bar with back button and player name
.career-icon-btn	<button>	The  icon in player column that opens career page
.modal-maximize	<button>	The  button in modal header to expand to career page
.weapon-track / .weapon-fill	<div>	Weapon breakdown bar chart; fill width set via inline style percentage

10. DOM Structure

10.1 Top-level Elements

ID / Selector	Tag	Purpose
#loading	div	Shown during CSV fetch; hidden on success
#error	div	Shown on fetch failure with error message
#content	div	Main app container; hidden until data loads
#statsBar	div	Row of aggregate stat cards
#awardsRow	div	Row of 4 award cards
#chartWrap	div	Top-10 killers bar chart wrapper
#chartBars	div	Bar chart inner container
.filter-panel	div	Filter controls (event type, search, pills, zeus)
#playerSearch	input	Player name search box
#missionSearch	input	Mission name search box
#playerPills	div	Player pill container
#missionPills	div	Mission pill container
#filterCount	span	Active filter summary text
#eventTypeSelect	select	Event type dropdown (Both / Joint Op / Regular Op)
#zeusFilterSelect	select	Zeus filter dropdown (All / No Zeus / Zeus Only)
#resetFilters	button	Resets all filters to defaults
#careerHeader	div	Career page header (hidden in leaderboard mode)
#careerPlayerName	div	Player name in career header
#careerPlayerSub	div	Missions · Active dates · Top role subtitle
#careerStats	div	Career page body content from buildCareerStatsHTML()
#infantryHead	thead	Infantry table header row
#infantryBody	tbody	Infantry table data rows
#vehicleHead	thead	Vehicle table header row
#vehicleBody	tbody	Vehicle table data rows
#playerModal	div.modal-overlay	Player detail modal overlay
#modalPlayerName	h2	Player name in modal header
#modalPlayerSub	div	Mission count / date range / role in modal header
#modalMaximizeBtn	button	Expand modal to full career page
#modalBody	div	Modal content (career HTML from buildCareerStatsHTML)

11. Deployment

11.1 GitHub Actions Pipeline

A GitHub Actions workflow deploys the application automatically on every push.

Branch Pattern	Deployed URL	Purpose
main	https://loltorres9.github.io/TFP_KillTracker/	Production
claude/*	https://loltorres9.github.io/TFP_KillTracker/dev/	Preview / development

11.2 GitHub Pages Configuration

GitHub Pages must be configured to serve from the gh-pages branch. Navigate to repository Settings → Pages → Source and select the gh-pages branch.

11.3 Publishing a Preview to Production

1. Develop on the claude/* branch.
2. Open a pull request from the claude/* branch into main.
3. Review and merge the pull request.
4. The GitHub Actions workflow triggers and deploys to production.

12. Extending the Application

12.1 Adding a New Stat Column

1. Add the raw column to the Google Sheet and publish the updated CSV.
2. In `buildAggregates()`, add an initialiser (e.g. `p.newStat = 0`) and an accumulator.
3. In `applyFilters()`, mirror the accumulation in the `tempAgg` block.
4. Add a column definition object to `INF_COLS` or `VEH_COLS`: `{ label, key, numeric, sortKey, render }`.
5. The table renderers will pick up the new column automatically.

12.2 Adding a New Award Card

1. Add an HTML award-card div in the `#awardsRow` section of `index.html`.
2. Give it a unique ID for each data element (name, stat, sub-stat).
3. In `renderLeader()`, sort/filter `filteredPlayers` to find the winner.
4. Write the winner's stats to the new DOM elements.

12.3 Pointing to a Different Google Sheet

1. Publish the new sheet: File → Share → Publish to web → Select sheet → CSV → Publish.
2. Copy the published URL.
3. In `tracker.js`, update the `CSV_URL` constant at the top of the file.

12.4 Adding Persistence (`localStorage`)

The application currently holds no persistent state. To save filter preferences across page loads:

- Write selected state to `localStorage` in `filterChanged()`.
- Read and restore state in `buildFilters()` before calling `refreshPills()`.
- Use `JSON.stringify/parse` for Sets.

Appendix A — Initialization Sequence

The following sequence occurs on every page load:

Step	Function / Event	Description
1	Page Load	Browser requests index.html; CSS parsed; tracker.js loaded.
2	Promise.all()	Parallel fetch of CSV_URL and unit_overrides.json.
3	parseCSVLine() × N	Each row parsed; rawRows[] populated.
4	buildAggregates()	aggPlayers{} populated with career totals.
5	classifyPlayerUnits()	playerUnits{} populated via co-occurrence analysis; overrides applied.
6	buildUI()	Calls buildFilters() and applyFilters(); shows section nav bar.
7	buildFilters()	Event listeners attached; refreshPills() called.
8	applyFilters()	filteredPlayers[] and filteredRows[] built; all render functions called.
9	renderStats()	#statsBar populated.
10	renderChart()	#chartBars populated.
11	renderLeader()	#awardsRow populated.
12	renderInfantryTable()	#infantryBody + #infantryHead populated.
13	renderVehicleTable()	#vehicleBody + #vehicleHead populated.
14	renderUnitLeaderboard()	#unitLbBody populated (section collapsed by default).
15	renderMissionHistory()	#missionHistBody populated (section collapsed by default).
16	renderVehicleKillsSection()	#vehKillsBody populated (section collapsed by default).
17	renderWeaponLeaderboard()	#wpBody populated (section collapsed by default).
18	renderMapStats()	#mapBody populated (section collapsed by default).
19	renderRoleLeaderboard()	#roleBody populated (section collapsed by default).
20	renderAttendance()	#attendBody populated (section collapsed by default).
21	renderKdTrend()	#kdTrendChart populated (section collapsed by default).
22	renderComparison()	Comparison dropdowns populated (section collapsed by default).
23	UI Ready	Loading spinner hidden; content and section nav shown.

Appendix B — Keyboard Shortcuts

Key	Context	Action
ESC	Modal open	Close the player detail modal
Click overlay	Modal open	Close the player detail modal

Appendix C — Browser Compatibility

The application uses standard ES6+ features: arrow functions, template literals, destructuring, Set, Map, fetch(), and async/await. It is compatible with all modern browsers (Chrome 80+, Firefox 75+, Safari 13+, Edge 80+). Internet Explorer is not supported.

Appendix D — ImportScript Reference

ImportScript is a Google Apps Script that reads OCAP .json.gz mission logs from Google Drive and writes one row per player per mission into the player_stats Google Sheet.

D.1 Required Google Drive Folder Structure

```
Kill Tracker/
■■■ OCAP_Logs/      ← drop .json.gz files here before running
■■■ OCAP_Logs_Archive/ ← processed files are moved here automatically
```

D.2 Installation

1. Open your Google Sheet.
2. Go to Extensions → Apps Script.
3. Paste the contents of ImportScript into the editor and save.
4. Run reimportOCAP to process one file from OCAP_Logs.

D.3 Entry Point

Function	Description
reimportOCAP()	Process (or re-process) the next file from OCAP_Logs; deletes any existing rows for that file before writing, making every run idempotent; archives the file to OCAP_Logs_Archive on completion

D.4 Unknown Entity Resolution

OCAP sometimes records entities without a name (e.g. players who disconnect before their name is synced). The script attempts to resolve these automatically:

- Unnamed entities with < 1 hour playtime are skipped entirely (reconnect ghosts, JIP artefacts).
- Remaining unknown entities are matched against named players who share the same group and role in the same mission.
- If exactly one named player maps to that group+role (after deduplicating candidates from reconnects), the unknown entity is renamed to that player.
- If a role string contains "@username" (e.g. "Rifleman@Zeus"), the script correctly detects it as a Zeus role.

D.5 Friendly Fire Counting

A kill is counted as a teamkill when victim.side === killer.side on a unit-kill event. It is never counted as a regular kill or weapon kill. Vehicle destructions are never counted as teamkills regardless of side.

D.6 End-of-Mission TK Suppression

Teamkills that occur during end-of-mission celebrations (players shooting each other or dropping grenades after the mission is won) are automatically excluded. Three overlapping heuristics are evaluated for each same-side kill event:

Heuristic	Condition	Rationale
Time window	TK occurs in the last 2 min (120 s) before endMission	Hard cutoff — mission is officially over
Close-range extended	Distance $\leq$ 15 m AND within last 3 min (180 s) of endMission	Grenade drops / pistol shots just before the end event fires
Burst detection	3 or more TKs fall within any 60-second rolling window	Group grenade throw or celebration volley earlier in the timeline

The endMission frame is read from the endMission event in the OCAP event stream; if absent, data.endFrame is used as a fallback. All thresholds are named constants at the top of the suppression block and can be tuned independently.

D.7 Sheet Schema

The script writes one row per player per mission. See §3.2 Expected Column Schema for the full column list. The sheet and header row are created automatically on first run; missing columns are migrated when the script is run against an existing sheet.

Changelog

A full history of changes — new features, fixes, and breaking changes — is maintained in CHANGELOG.md at the root of the repository.

GitHub: https://github.com/loltorres9/TFP_KillTracker/blob/main/CHANGELOG.md